

Prime Agent against rung-agent

Prime Intellect's open-source agent harness, read next to ours. What it does better, what we do better, and what is worth borrowing.

1. **Prime's biggest lead is isolation and durability.** Every session runs in its own worker process under a small supervisor, with an on-disk log. Our **rung-agent --acp** serves every session from one process behind one queue, and its session file is written only when a turn starts and ends [P1, P3, P4, R1, R3].
2. **Our long sessions never shrink.** Prime compacts at a threshold; rung-agent replays all history and has one emergency step: drop the oldest tool results once [P9, R5]. It also does not know the model's window size [R7].
3. **Prime's verifiers are the idea worth taking, not its code.** It checks each release against a pinned earlier binary. Its performance and parity harnesses were deleted from the repository after the port [P20, P21].
4. **The "RLM" is a persistent Python REPL with a bridge to the host.** Nothing in the repository shows it beats a plain tool loop on long context. We already own half of it, so test before building [P10, P11, R11].
5. **Licence is MIT.** Ideas and small pieces may be borrowed with the notice kept. I recommend ideas only. The continuous host cannot run prime-agent as its engine. Six questions for the owner are on pages 16 and 17.

Both code bases were read, not their READMEs

Every claim below carries an evidence id. Verified means I read it in code or in git history. Claimed means a document says so. Inferred is my reasoning.

What I read

- **Prime:** README, house rules, the blog text, all nine crate READMEs, the agent loop, supervisor and worker code, journals and leases, compaction, the ACP and MCP code, the model catalogue, the Python REPL runtime and its protocol document, the factory skill, the tests and CI files, 56 change notes, and git history for the benchmark tooling.
- **Rung:** house rules, the agent library and CLI, the turn engine, ACP and MCP, sessions, the loop and sandbox in rung-std, the memory crate, the whole continuous-host design, its record and its gates.

Size, for scale (verified by counting)

	PRIME	RUNG
Rust lines, tests included	565,000	92,500
Python runtime lines	11,800	about 200 (sandbox guest)
Test functions	about 5,300	about 830
Commits	5,050	not counted

Much of prime's size is the price of byte-for-byte parity with its TypeScript predecessor. Size is not a score.

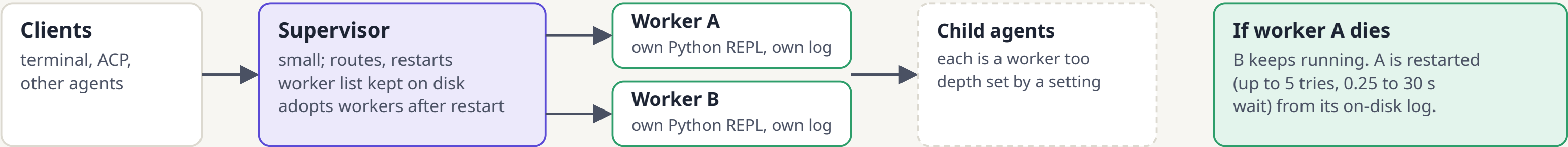
What I did not verify

- I ran nothing: no build, no test, no timing. All speed and memory figures are prime's own claims.
- I did not read the terminal UI crate (137,000 lines) beyond its test names, nor the 4,700-line factory module beyond its documents.
- I did not read the research paper. The blog's agent counts (2,209 agents, 228 billion tokens) are claims.
- Whether the Python REPL style beats a plain tool loop: no ablation is in the repository.
- Whether the model router the host already queries reports window sizes: inferred, not read.

Where a session lives: prime isolates it, rung-agent shares it

The single most important structural difference. The same two questions are asked of each: what happens to the others when one session is busy, and what happens when it dies?

prime-agent: one process per session



rung-agent --acp: every session in one process



The continuous host (rung-host) is a different shape: one agent, one fsynced record, restarted by its service manager. It is not drawn here and is not affected by the queue.

Loop, state, sessions, process, protocol and tools

Prime is a platform around one tool. Rung is a small set of typed parts with several tools. Neither is a superset of the other.

DIMENSION	PRIME-AGENT	RUNG (RUNG-AGENT AND RUNG-HOST)	EVIDENCE
Loop	A tool-agnostic turn loop with three message sources: steering, follow-up and continuation hooks. No cap on model calls per prompt and no repeat guard that I found; the opt-in autonomous mode stops at 12 turns.	The loop is a typed ladder: Calling, then tool results, then EndTurn. Call caps per kind, a last-step "answer now" nudge, and a stop after the same call fails to progress three times.	P7 P8 R6
State	An append-only log per session, a recovery journal fsynced on every write, worker records on disk, a lease per session file. Old sessions move to an archive by age and count [P28].	Agent sessions: one JSON file rewritten whole, saved when a turn starts and ends, no fsync, no lock. Host: one append-only fsynced record, torn last line cut on open, gap-free sequence.	P4 P5 P6 P28 R3 R4 R12
Sessions	Resumable, forkable, archived, listed in an agents view; saved sessions wake on a message.	Resume, fork, list, close over ACP; background runs detached with a pid. One agent in the host, with channels.	P1 R4 R17
Process model	Supervisor, one worker per session, a Python REPL per worker; restart with backoff; adopt live workers after a supervisor restart.	Agent: one process for all ACP sessions, one queue, one working directory. Host: one process, service-manager watchdog, recovery by replay, 50 random kill -9 gate.	P2 P3 P6 R1 R2 R13
Protocol	ACP over stdio only, served through the daemon. No load, no list, no resume. Also a private daemon protocol shared by all crates.	ACP over stdio and streamable HTTP with load, list, delete, close, mode, resume, fork. The host adds channels, status and stimulus extensions.	P14 P16 R17
Tools	One model tool, a Python REPL. Shell, edit, web, MCP, subagents and skills are Python functions inside it. Not a sandbox.	Groups chosen per run: read, write, shell, web, todo, skill, task, optional Python. The Python guest runs in a bubblewrap jail with the network off. The shell tool is not jailed.	P10 P23 R11

Context, subagents, models, extension points, language and verification

Here prime is ahead on volume of machinery; rung is ahead where the machinery is a proof rather than a convention.

DIMENSION	PRIME-AGENT	RUNG (RUNG-AGENT AND RUNG-HOST)	EVIDENCE
Context and compaction	Compacts when context passes a threshold: keeps the last 20,000 tokens, writes a structured summary, keeps the Python state, drops variables over 16 MiB. Cache-stable prompt layers plus one changing tail.	Agent: no compaction. Host: epochs. The pack is stable layer, slow layer, then an append-only log; roll over at 60 to 85 percent of the budget; every request extends the last byte for byte (a gate).	P9 P10 R5 R15
Subagents	Real child agents, each its own worker; spawn returns a handle at once; collect, list, delete, rename, progress notes; depth is a setting; parent, sibling and child messaging; results arrive as messages.	A nested task tool: depth 1, run synchronously, parent waits. The host reserves a slot for workers, to be built last.	P12 P13 R10 R18
Models and catalogue	A catalogue fetched at runtime (hourly, on picker open, on login), validated, with a last-good disk cache, a bundled snapshot and a compiled fallback. Transport addresses are pinned in code. Carries window, output and price.	One endpoint and model from config or environment. The host lists the router's models every 6 hours, keeps the free ones, and walks a compiled ladder of five names. The window size is unknown to ACP clients.	P17 R7 R8 R9
Extension points	Skills are importable Python packages; MCP runs inside the REPL with tool search; harness entries (memories, prompt notes, skills, subagent specs) refined by a model with history and rollback; a factory of state-machine workflows (off by default).	Memory is a provider contract with typed outcomes; MCP tools join the roster; a decision desk with swappable deciders; reserved crew slot. Skills are plain files.	P18 R16 R17
Language and safety	Rust, nine crates, a one-way dependency graph, unsafe code forbidden, strict lints, dependency advisory and licence gate. README says it is not a security sandbox.	Rust, a proc-macro that refuses skipped steps at compile time (57 pinned refusals), small crates. No dependency advisory job in CI. Python guest jailed; shell not.	P23 P24 R19 R21
Verification	Merge gate: a diff against the previous product, a recorded-corpus replay, user-level terminal tests. Release-time comparison to a pinned binary. Perf tooling removed from the repository.	Frozen numbered gates for the host (G-a to G-r), seeded and free. Memory scoring harness. Validation suite on Harbor tasks. No release-to-release diff, no performance baseline for the agent.	P19 P20 P21 R13

Eight things prime has that rung lacks, ranked by value to rung

Each row names a harm that can be measured, not a matter of taste. The class column says what kind of problem it is an instance of.

#	GAP	CONCRETE HARM IN RUNG TODAY	MEASURABLE GAIN	CLASS OF PROBLEM	EV.
1	One process per session	While session A runs a long turn, B and C wait for it. A crash, out-of-memory or hang in any turn stops all sessions. The process-wide working directory makes parallel turns unsafe.	Time for B's first update while A runs 60 s: now about 60 s. Others alive after killing one worker: now none.	Shared fate in one process	P2 P3 R1 R2
2	Compaction and a known window	History replays every turn and only grows. At overflow one step drops old tool results once; a second overflow ends the turn. ACP clients see window size 0 and cannot warn.	Turns survived in a 200-turn scripted session; recall of an early fact after compaction; window size non-zero on every call.	Unbounded growth of long-lived state	P9 R5 R7
3	Durable session log and lease	A crash mid-turn loses every step of that turn. Two writers to one session id: last write wins. Whole-file rewrite grows with history.	Steps kept after kill -9 mid-turn: now none; lost updates in a two-writer test: now possible.	Durable state with weak write rules	P4 P5 R3 R4
4	Reference verifier between releases	A change to prompts, tools or serialisation can alter the bytes sent to the provider, or break the cache prefix, unseen until a live run.	Count of unintended request-byte differences per release; target zero.	Verification with no external reference	P19 P20
5	Performance baseline and gate	No measurement of start-up, memory at large sessions or session switch; regressions arrive silently. Prime's gate tripped at 10 percent.	Cold start, memory at a 10 MiB session, switch latency, each with noise bounds.	Unmeasured quality attribute	P22
6	Asynchronous subagents	The parent is blocked while a child runs; no parallel fan-out; no way to steer a running child; depth fixed at one.	Wall time of a three-way fan-out against three sequential tasks.	Blocking composition	P12 R10
7	Catalogue-driven model facts	New models and window sizes need a code change or a hand-edited config; the host's ladder is a compiled list.	Time from a model's appearance to use; fewer releases for catalogue changes.	External facts baked into code	P17 R9
8	The REPL with a host bridge	Rung's Python guest cannot call tools or spawn agents, and has no background tasks. Bulk data stays in the transcript unless the model scripts around it.	Tokens per solved long-input task, if the experiment on page 9 shows a gain.	Context held in the transcript	P10 P11 R11

Prime recovers a dead session from disk; rung-agent does not try

The host already has the recovery design. The agent library and CLI do not use it.

PRIME how it works

- The supervisor spawns one worker per active session, restarts a dead one with a wait of 0.25 s growing to 30 s, gives up after 5 consecutive failures, and treats 30 s of life as proof of health [P2, P3].
- Worker records sit on disk, so a restarted supervisor adopts live workers or relaunches them [P6]. One process may hold a session file, enforced by a lease with stale-owner reclaim [P5].
- Queue and busy state go to an append-only journal, fsynced on every write [P4].
- A hard-killed parent's children are closed by the supervisor; tested, but the test skips silently when no kernel is installed [P27].

Not to copy

- The size of the machinery (156,000 lines in the daemon crate) is the cost of a private wire protocol kept byte-compatible with a predecessor.
- Skip-when-missing tests: a verifier that skips is not a verifier.

RUNG-AGENT how it works

- All ACP turns run through one queue and set the process working directory before running [R1, R2].
- A session is one JSON file, rewritten whole through a temporary file and a rename, with no fsync and no lock [R3]. It is saved when a turn starts and when it ends [R4].
- A session that was running with a dead pid is marked interrupted. Nothing replays it [R4].

RUNG-HOST already does it

- One append-only record, written per event, fsynced when a stimulus is accepted and a turn ends. A torn last line is cut on open [R12].
- Replay rebuilds the state; an unfinished stimulus is requeued; 50 random kill -9 runs must give each stimulus exactly one outcome [R13].

CLASS REMEDY make the unit of failure the unit of work, and make the log the truth. Rung owns both techniques; they are not applied to the agent path.

A long rung-agent session ends in an overflow with no durable cure

Prime and the host each bound context in different ways. The agent library has neither.

PRIME summary compaction - Reserve 16,384 tokens; keep the latest 20,000; when context passes the threshold a model writes a fixed-format summary (goal, progress, decisions, next steps) and later updates it in place [P9]. - The Python state survives; after compaction the model is told which names remain and which were dropped for size [P10]. - The cost: the whole cached prefix is lost at every compaction, and the summary is the model's own account of what mattered. No document I read measures what it loses.	RUNG-HOST epochs - Stable layer, slow layer, an append-only log. Roll over at the first break past 60 percent, always at 85, never below 40. Text the open work refers to is kept word for word [R15]. - Every request extends the one before it byte for byte, so a provider's prompt cache holds; a gate runs 10,000 turns [R15]. - The budget is a fixed number, 128,000 tokens, not read from the model [R9].	RUNG-AGENT nothing durable - Each turn replays every earlier turn's full tool calls and results [R4]. - On a provider overflow the oldest half, by weight, of tool results is replaced by a note, once per turn; a second overflow ends the turn [R5]. - ACP usage updates carry window size 0 unless a refusal states it, so a client cannot show "80 percent full" [R7].
--	---	--

Reading

The harm is concrete: a long ACP or CLI session reaches the window, pays for a one-off elision, and fails on the next. The remedy is a bounded-state step with a recorded outcome. The host's rollover is the better model for rung than a model-written summary, because it keeps the cache and records what was evicted. Both need the window size first.

What the RLM is, and whether it is better for long context

It is a persistent Python REPL as the only model tool, with a bridge back to the harness. It is a way to hold context outside the transcript. Whether it wins is not shown in this repository.

What it is (verified)

- The model sees one tool, **ipython**. Everything else (shell, edit, web search, MCP, subagents, memory, goals, schedules) is an async Python function in the same namespace [P10].
- The REPL is a real process: one namespace on one asyncio loop, cells with top-level await, background tasks that outlive a cell, interrupts, and a request channel to the host so Python code can ask the harness to spawn a child or call MCP [P11].
- State is saved with dill, name by name, with size caps; oversized names are pruned and reported; restore revives each name on its own [P11].
- "Prompt as a variable" is a prompt rule, not a mechanism: the system prompt tells the model to assign results to variables and keep large data on disk [P10]. I found no code that loads a long input into a variable for it.

What rung already has (verified)

- A Python sandbox whose namespace is pickled between strikes, usable as a tool or as the only action channel (inline mode) [R11]. It has no host bridge, no async, no background tasks, no tools inside.

Is it better for long context? Honest answer

- **Plausible mechanism:** bulk data stays in variables, so each turn's cost grows with what the model prints, not with what it read.
- **Plausible costs:** the model must write correct code on every step; a crash loses the live state unless it was snapshotted; names over 16 MiB are dropped at compaction [P10].
- **Evidence in the repository:** none. The blog's numbers measure the Rust rewrite's speed, not the REPL's accuracy.

RECOMMENDED TEST

Run rung's existing inline-Python mode against its normal tools on long-input cases in the existing Harbor suite. Measure pass rate, peak context and tokens per solved task. Build a host bridge only if inline mode wins.

CLASS context held in the transcript instead of an addressable store. The class remedy is handles: data lives outside, the transcript holds names and small views.

What prime's verifiers really are, and what survived

The blog credits parity checks and a performance loop. In the repository, most of that tooling is gone.

Still in the repository (verified)

- A merge rule: verifiers over self-assessment; parity evidence against the previous product is a gate [P25].
- A differential test of CLI output against the installed old binary, skipped when it is absent [P19].
- A CI step that downloads one pinned old release, checks its hash, and compares lifecycle behaviour, with receipts; and a shard audit that fails if any selected test was silently dropped [P20].
- A frozen corpus of the old tools' outputs replayed at test time, and terminal-mode tests over a real pty [P29].

Removed from main on 30 September (verified)

- The performance battery, the parity harnesses it fed, the recorded baseline and the scripted-model driver, as unused after the port. The "benchmark campaign harness on the bench node" is out of tree [P21].
- I found no terminal frame-diff harness in the tree, so the blog's frame comparison could not be inspected.

The method, from history (verified)

- Run old and new binaries side by side, interleaved, medians of several trials, against one deterministic scripted model, on one quiet machine [P22].
- Track start-up, typing latency, memory with sessions open, resume of a large transcript, first tool output, streaming, compaction time, export time, idle CPU. Gate: more than 10 percent worse than the recorded baseline fails [P22].
- The blog adds: withhold results too noisy to compare; agents profile, propose, A/B on the same machine, two reviewers on different models; no numeric target.

Self-improvement, in two layers

- **In the product:** a model proposes small edits to memories, prompt notes, skill descriptions and subagent specs, with a history and snapshots for rollback [P10]. Narrow and reviewable.
- **Around the product:** an orchestrator that writes no code, with planner, implementer, reviewer and verifier as separate agents. This is the part that did the work, and it needs a verifier first. Rung has the equivalent machinery and has dispatched no real judgment yet [R20].

CLASS verification with no external reference. The remedy is a pinned prior artefact plus recorded inputs, and a measured baseline with noise bounds.

Where rung is ahead, stated plainly

Several of these are properties prime reaches by convention and tests. In rung the compiler or a numbered gate holds them.

A turn that is bounded and honest

Call caps per kind, a last-step nudge, a stop on three identical failed repeats, a typed failure kind that separates overflow from refusal [R5, R6]. I found none of these in prime's loop; its only turn limit is the opt-in autonomous mode [P7, P8]. Harm avoided: a runaway loop on a paid model.

Memory as a contract

A provider declares capability and budget; recall and retain end in typed outcomes; an unreachable provider can never be read as an empty one; scoring harness on real notes [R16]. Prime's memory is entries the agent edits, with versions and rollback, and no outcome type that separates absent from failed.

The state machine is the type

A skipped rung is a compile error, with 57 committed refusal pins [R19]. Prime has a legal-transition table for its update flow, checked by tests, and exhaustive enums by house rule [P25]. Harm avoided: an unreachable state reached by a refactor.

A real jail for the code tool, and a wider ACP

The Python guest runs under bubblewrap with the network off [R11]. Prime states it is not a sandbox [P23]. Rung's ACP serves HTTP and loads, lists, resumes and forks sessions; prime's serves stdio only and cannot load [R17, P14].

A host that never rests, proven to restart

Prime has heartbeats, schedules, goals and a bounded autonomous mode, all of which re-enter a chat session. It has no equivalent of a host that owns time, a gap-free record, replay equality by hash, a free-time mode, a cache gate, or 50 random kill -9 runs [R12, R13, R15].

Small enough to hold in mind

92,500 lines against 565,000. Partly an unfair comparison, since prime carries a parity burden. **Against us:** the shell tool is not jailed, there is no dependency advisory gate, and the audit loop's defect count is still zero [R21, R20].

Ten ideas, sized, with risk and what to measure

S is days, M is a few weeks, L is a quarter. Every idea is rung's own implementation. Licence column: I = take the idea only.

#	IDEA, IN RUNG TERMS	SIZE	RISK	MEASURE	LIC.
1	Reference verifier: run the scripted inputs on the previous tagged release and the candidate; diff provider request bytes and saved sessions in CI, pinned by hash	S	False alarms on intended change: require a written expected-diff note.	Unintended request-byte differences per release (target 0); cache-prefix breaks.	I
2	Performance baseline and gate for rung-agent: interleaved trials, medians, noise check, 10 percent threshold	S	Noisy machines; withhold unstable results.	Cold start to first request; memory at a 10 MiB session; session switch time.	I
3	Model facts table (window, output, price) from the router listing the host already fetches, plus a local override and a last-good cache	S	Wrong numbers cause early rollover. Catalogue may carry facts only, never addresses or headers.	Window size on 100 percent of usage updates (now 0).	I
4	One worker process per ACP session, reusing the existing detached-child code, with restart and backoff	M	Cancel and streaming across a process boundary; more processes.	B's first update while A runs 60 s; sessions alive after kill of one.	I
5	Append-only fsynced session log with a single-writer lease, reusing the host's record design; old files stay readable	M	Format migration; lease reclaim after a crash.	Steps kept after kill -9 mid-turn; lost updates in a two-writer test.	I
6	A typed compacting step for rung-agent: threshold, rollover or summary, recorded outcome; start from the host's epoch policy	M	A summary can lose facts; one cache miss per rollover.	Survival of a 200-turn scripted session; recall of an early fact; cache hit rate.	I
7	Experiment: inline-Python mode against tools on long-input Harbor cases	S	Little: it uses code that exists.	Pass rate, peak context, tokens per solved task.	I
8	Dependency advisory and licence gate (cargo-deny) in CI	S	Triage of existing advisories.	CI fails on a known-bad or yanked dependency.	I or C
9	Asynchronous subagent handles as a typed ladder: Spawned, then Settled, and a handle cannot be dropped unsettled	L	Largest; collides with "workers last" in the host plan and the shared working directory.	Wall time of a three-way fan-out against sequential tasks.	I
10	Make skipped verifiers loud: a counted skip with a reason, and a gate that fails if an expected gate did not run	S	Little.	Skipped-gate count in CI output (target 0 on the gate box).	I

Prime Agent is MIT. Take the ideas; copy code only if it is worth the notice.

Checked in the file itself [P26]. Rung's own manifest says MIT or Apache-2.0, so the two are compatible.

What the file says (verified)

- MIT, copyright 2025 to 2026 Prime Intellect Ltd. Use, copy, modify, merge, publish, distribute and sublicense are all permitted [P26].
- One condition: the copyright and permission notice must stay in all copies or substantial portions.
- A second notice follows: the software is a port of the earlier TypeScript product, originally copyright 2025 Mario Zechner, under MIT [P26]. A copied file would carry both.
- No patent grant and no warranty. Apache-2.0 would have given a patent grant; MIT does not.

My recommendation

- **Borrow ideas, not code.** The code is bound to a private wire protocol and to byte parity with its predecessor, so little of it lifts cleanly.
- Code worth reading, if one wants a second view: the backoff and stable-lifetime rule [P3], the lease [P5], the shard audit [P20]. Copying any of them is allowed with both notices kept in the file.
- The summary prompts [P9] read like earlier open work. Write rung's own.
- Nothing from prime needs a rung change to its licence field.

Standalone rule

Rung takes no dependency on prime-agent or any consumer. Nothing above adds one.

Could the continuous host run prime-agent? Not as its engine

The host owns the thread and the tools. Prime owns its own. A narrower role, as a delegated worker, is possible later.

Why not as the engine (verified)

- The host's engine seam is: the host builds the whole thread and tool set; the engine runs one bounded turn and reports every model call. The caller owns the thread [R14].
- Through ACP, prime takes a prompt and keeps its own history, summary compaction, system prompt and tools; its only model tool is the REPL [P10, P14]. The host could not hold its byte-for-byte cache gate, its tool gate or its rollover.
- Prime's ACP emits tool calls, but shows shell, REPL and MCP all as one execute kind [P14]. I found no usage updates in its ACP code, so per-call cost records would be missing.
- The host has no ACP client; it serves ACP only [R22]. A client engine would be new code.

Interop facts that matter

- **Resume:** prime advertises no load and no list, so the host could not reopen a prime session after its own restart through ACP [P14].
- **MCP:** prime advertises HTTP only, though its code also admits stdio servers [P14, P15]. Rung's memory provider over HTTP would probably reach it (inferred); over stdio is not advertised. MCP calls happen inside the REPL, not as ACP tool calls.
- **Weight:** an ACP process attaches to a daemon with a supervisor, a worker and a Python REPL environment [P16, P2].

POSSIBLE LATER as a crew worker

The host reserves a crew slot and admits external completion items [R18]. A prime worker could take delegated jobs over ACP, with the host recording each handover and result. This is a later extension and needs the ACP client first.

Each gap is an instance of a class; fix the class

The remedy for a class applies to the next instance as well. Prime's own change notes are full of fixes of these kinds (a supervisor eviction fence, a kernel wedge on restore, stale session leases).

#	GAP	CLASS OF PROBLEM	CLASS-LEVEL REMEDY	WHERE RUNG ALREADY HAS THE TECHNIQUE
1	Sessions share one process	Shared fate: one failure domain for unrelated work	Make the unit of work the unit of failure; supervise and restart; keep the state outside the process.	Detached child with pid; host's watchdog
2	History only grows	Unbounded growth of long-lived state	A budget, a trigger and a recorded, typed rollover. State the window first.	Host's pack epochs
3	Session lost mid-turn	Durable state with weak write discipline	Append-only log, fsync at commit points, one writer by lease, replay to rebuild.	Host's record
4	Window, price and ladder in code	External facts baked into code	Facts as validated data with provenance and a last-good cache; never let data carry addresses or headers.	Host's router listing
5	No cross-release check, no perf baseline	Verification without an external reference	Pin a prior artefact by hash; replay recorded inputs; diff outputs; keep a measured baseline with noise bounds; make skips loud.	Frozen numbered gates in the host
6	Synchronous subagent	Blocking composition	An asynchronous handle that must be settled, so work in flight cannot be forgotten.	Typed ladders with must-use tokens
7	Bulk data in the transcript	Context in the wrong store	Handles to data kept outside; test the benefit before building a bridge.	Python guest, inline mode
8	No advisory gate	Supply chain seen only after the fact	A gate in CI, with reasons and review dates on every exception.	None

Three questions, most blocking first

Each has a recommendation and states what the answer changes. Evidence is on the previous pages.

1. Put verifiers first: a release-to-release request diff and a performance baseline for rung-agent?

Before any of the changes below, should rung-agent get the two checks that tell us whether they helped? The first replays scripted inputs on the previous tag and the candidate and diffs the provider requests and saved sessions. The second records start-up, memory at a large session and session switch time, with noise bounds.

Recommendation. Yes, both, small (ideas 1 and 2). Without them the next three changes cannot be measured, and a cache-prefix break would go unseen.

What the answer changes. Whether work starts with measurement (days) or goes straight to isolation; CI time rises by one scripted run per release.

2. Give each ACP session its own worker process?

Should the --acp server start one detached child per session, with restart and backoff, in place of one process with one queue?

Recommendation. Yes, after question 1 (idea 4). It removes the head-of-line wait and the shared working directory, and it contains a crash to one session. Keep the host as it is.

What the answer changes. How cancel and streaming cross the process boundary; per-session memory cost; the process-wide working directory stops being a constraint.

3. Give the agent path the host's durable record?

Should rung-agent sessions move from one JSON file written at turn start and end to an append-only fsynced log with a single-writer lease, reading old files as before?

Recommendation. Yes, together with question 2 (idea 5). A crash then keeps the steps already taken, and two writers cannot lose an update.

What the answer changes. The saved-session format and its migration; resume after a crash replays the log instead of marking the session interrupted.

Three more, about context, the RLM and the host

These depend on the first three only in order, not in substance.

4. Add model facts and a typed compacting step to rung-agent?

Should the window size come from the router listing the host already fetches, then drive a compacting step that borrows the host's rollover policy rather than a summary-only design?

Recommendation. Yes, in two stages (ideas 3 then 6): facts first (days, makes ACP usage updates honest), compaction after (weeks, measured on a 200-turn scripted session).

What the answer changes. Whether long agent sessions survive; one cache miss per rollover; a new typed rung and its refusal pins.

5. Test the RLM idea before building any of it?

Should we run rung's existing inline-Python mode against its normal tools on long-input Harbor cases, and build a host bridge or asynchronous subagents only if it wins?

Recommendation. Yes, as an experiment (idea 7). Do not build the REPL bridge or asynchronous subagents (idea 9) on faith; the host plan already puts workers last.

What the answer changes. Whether the largest item on the list (asynchronous subagents) is ever started, and whether the Python guest gains a bridge.

6. Run prime-agent under the continuous host?

Should we plan to run prime-agent as the host's engine, or later as a crew worker?

Recommendation. Not as the engine: the host's cache, tool and rollover gates need the thread. As a crew worker, revisit after the crew extension exists and an ACP client is written.

What the answer changes. Nothing now. It keeps the host's gates intact and avoids a second long-lived daemon on the box.

Also, one small decision I made

I put a dependency advisory gate (idea 8) and loud skips (idea 10) on the list but did not make them questions: both are small, reversible and carry no design choice.

Prime Agent evidence, with file and line (1 of 2)

Paths are relative to the Prime Agent repository. Line ranges are from commit 9fcfbf02c.

ID	WHERE	WHAT IT SHOWS
P1	AGENTS.md:205-207	Daemon is a supervisor; one worker process per session; restart with backoff; append-only logs; reattach after supervisor restart
P2	crates/pa-daemon/src/supervisor.rs:1-4	Same, in code: worker per session, restart with backoff, descriptors persisted
P3	crates/pa-daemon/src/supervisor/supervision.rs:15-21	5 consecutive failures; 250 ms to 30 s backoff; 30 s of life counts as healthy
P4	crates/pa-daemon/src/journal.rs:11-27	Append-only recovery journal; fsync after each write
P5	crates/pa-daemon/src/lease.rs:1-5	One process per session file; lease with stale-owner reclaim
P6	crates/pa-daemon/src/descriptor.rs:1-5; supervisor/adoption.rs:1-3	Worker records on disk; new supervisor adopts or relaunches
P7	crates/pa-agent/src/agent_loop/run.rs:21-259 (context clone at :128)	Turn loop with steering, follow-up and continuation hooks; no call cap or repeat guard found by search
P8	crates/pa-core/src/autonomous/mod.rs:23	Default 12 turns, in opt-in autonomous mode only
P9	crates/pa-core/src/session_engine/compaction.rs:7-8,166-180,434	Reserve 16,384; keep 20,000; threshold test; summary prompt
P10	crates/pa-core/src/prompts/layers/core.md:1-3,139; layers/usage.md:11-18; AGENTS.md:174,176	One model tool (REPL); state survives compaction; variables over 16 MiB dropped; data-on-disk advice; self-refine rules; cache-stable prompt layers plus one dynamic tail
P11	prime-agent-runtime/src/rlm/repl.md:98-110,151-175	One namespace on one asyncio loop; host request bridge; dill snapshots with size caps
P12	crates/pa-core/src/prompts/layers/core.md:27-52	spawn, collect, list, delete, rename, progress notes; results by message
P13	crates/pa-core/src/settings/load.rs:21,130	rlmMaxDepth setting
P14	crates/pa-daemon/src/acp/types.rs:86-93,386-398,421-426	Capabilities: no load, image and embedded context, MCP http, close only; tool kinds

Prime Agent evidence, with file and line (2 of 2)

Continued. P21 and P22 are git history, not files on the main branch.

ID	WHERE	WHAT IT SHOWS
P15	crates/pa-daemon/src/acp/mcp.rs:18-27,143,260	Stdio and http MCP servers admitted on session/new
P16	crates/pa-daemon/src/acp/daemon.rs:1-12,1021-1036	ACP served over a daemon session; handshake methods
P17	crates/pa-models/README.md:1-30; src/fetch.rs:8-13; src/lib.rs:21	Runtime catalogue; last-good cache; bundled and compiled fallbacks; pinned transports; hourly refresh
P18	skills/factory/SKILL.md:1-30	State-machine workflows of child agents; ships disabled
P19	crates/pa-cli/tests/differential_cli.rs:12-17	CLI differential against old binary; skipped when absent
P20	ci.yml (under .github/workflows):335-352; scripts/ci_test_shard_summary.py:1-25	Pinned, hash-checked old release; shard audit against dropped tests
P21	git commit 583b60311, 2026-09-30	Perf battery, parity harnesses, baseline removed from main; bench harness out of tree
P22	git show 583b60311^: scripts/battery/perf_gate.py:1-14; perf_wave.py:1-30	Interleaved medians; same scripted model; 10 percent gate
P23	README.md:87	Worker and kernel processes are not a security sandbox
P24	Cargo.toml:48-49	Unsafe code forbidden at workspace level
P25	AGENTS.md:71,130-142	Verifiers over self-assessment; parity evidence as a merge gate
P26	LICENSE:1-25	MIT, Prime Intellect 2025-2026; port notice for the TypeScript product, 2025 Mario Zechner, MIT
P27	crates/pa-daemon/tests/rlm_children_parent_death_e2e.rs:5,99	Test skips when no kernel is installed
P28	crates/pa-daemon/README.md:33,43	Session archiving by age and count; saved sessions wake on a message
P29	crates/pa-core/src/tools/golden_replay.rs:1-9; crates/pa-cli/tests/terminal_state_differential_e2e/main.rs:12-17	Frozen corpus from the old tools replayed; terminal modes checked over a real pty

rung evidence, with file and line

Paths are relative to the rung repository at 085933d. R21 and R22 record an absence, found by search, not a line.

ID	WHERE	WHAT IT SHOWS
R1	rung-agent-core/src/acp.rs:14-17	One process-wide FIFO; turns serialised; process-global working directory
R2	rung-agent-core/src/acp.rs:1139-1147	Turn on a blocking thread after set_current_dir
R3	rung-agent-core/src/session.rs:186-195	Session saved as one JSON file via temp file and rename; no fsync, no lock
R4	rung-agent-core/src/run.rs:597,657,840	Saved at turn start and end; dead pid marked interrupted; turn history replayed (:438)
R5	rung-std/src/agent.rs:358	Oldest tool results elided once on overflow; one-shot per turn
R6	rung-std/src/agent.rs:93-99,150; rung-agent-core/src/catalog.rs:59	Doom streak 3; last-step nudge; call caps per kind
R7	rung-agent-core/src/acp.rs:596,686	usage_update window size 0 unless a refusal states it
R8	rung-agent-core/src/config.rs:15-16; engine.rs:68	One endpoint and model from config and environment
R9	rung-host/src/ladder.rs:1-20,30; core.rs:104	Router listing every 6 h; compiled ladder; epoch budget 128,000
R10	rung-std/src/tools/task.rs:9-14; rung-agent-core/src/background.rs:1-2	Nested task depth 1; synchronous spawn
R11	rung-std/src/python/mod.rs:1-6; jail.rs:83-84; guest/guest.py:4	Stock CPython child; pickled namespace; bubblewrap; network off
R12	rung-host/src/record.rs:1-18	Append-only fsynced record; torn-line cut; gap-free sequence
R13	docs/rung-host.md:349-366,423	Watchdog; replay; 50 random kill -9 gate
R14	rung-host/src/engine.rs:1-9; adapter.rs:5-9	Host assembles the thread; caller owns the thread
R15	docs/rung-host.md:104-110,425	Byte-prefix requests; cache gate
R16	rung-memory/src/lib.rs:1-30	Typed recall and retain outcomes
R17	rung-agent-core/src/acp.rs:1-5; mcp.rs:1-4	ACP surface; stdio and http MCP client
R18	docs/rung-host.md:12-15	Delegation is a final extension; crew slot reserved
R19	rung/tests/spec_refusals.rs:15; 57 .stderr files	Compile-time refusals pinned
R20	AGENTS.md:237	Audit-rectify defect count still zero
R21	(absence) no deny.toml; .github/workflows has ci.yml, review-council.yml	No advisory job
R22	rung-host/src/acp.rs:1-30	Host serves ACP as agent; no ACP client code found by search